

Al-Aqsa University
Faculty of Computers and IT
Engineering and Networks Dept.



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم الهندسة والشبكات

Advanced Algorithm

Answers of Assignment **2** : Divide and Conquer

Presented to: **Eng. Firas Foad AL-Ejla**

Student name: **Deema Mohammed AL-Maqadma**

Student number: **2320230766**

Semester: Second Semester 2024/2025

السؤال الأول:

Divide and Conquer خوارزميتان من

1. خوارزمية ترتيب الدمج (Merge Sort)

1. فكرة الخوارزمية /

- تقسم المصفوفة إلى نصفين متساويين (أو متقاربين) حتى تصبح كل قطعة تحتوي على عنصر واحد أو عنصرين
- ترتب كل قطعة صغيرة بشكل فردي
- تدمج الأجزاء المرتبة معًا بترتيب تصاعدي أو تنازلي
- Divide and Conquer تعتمد على مبدأ

2. تطبيق الخوارزمية يدويًا على المصفوفة

[8, 3, 10, 1, 6, 4, 14, 7, 13]:

الخطوات:

1. تقسيم المصفوفة:

[8, 3, 10, 1, 6] | [4, 14, 7, 13]
[8, 3] | [10, 1, 6] | [4, 14] | [7, 13]
[8] | [3] | [10] | [1, 6] | [4] | [14] | [7] | [13]
[يتم دمجها لتصبح ٦، ١] ← [1, 6]

2. الدمج مع الترتيب:

[دمج ٨] و [3] ← [3, 8]
[دمج ١٠] و [١، ٦] ← [1, 6, 10]
[دمج ٤] و [١٤] ← [4, 14]
[دمج ٧] و [١٣] ← [7, 13]
[دمج ٨، ٣] و [١٠، ٦، ١] ← [1, 3, 6, 8, 10]
[دمج ١٤، ٤] و [١٣، ٧] ← [4, 7, 13, 14]

النتيجة النهائية:

[1, 3, 4, 6, 7, 8, 10, 13, 14]

3. تحليل التعقيد الزمني :

تعقيد التقسيم:

لأن المصفوفة تنقسم إلى نصفين في كل مرة ($O(\log n)$).

تعقيد الدمج:

لأن كل عنصر تتم مقارنته مرة واحدة أثناء الدمج ($O(n)$).

إجمالي التعقيد:

(في جميع الحالات (الأسوأ، الأفضل، والمتوسط ($O(n \log n)$)).

تطبيق الخوارزمية عمليا بكود بلغة الجافا /

```
public class MergeSort {
// Deema Mohammed AL-maqadma
    public static void mergeSort(int[] arr) {
        int n = arr.length;
        int mid = n / 2;
        if (n <= 1) {
            return;
        }
        int left[] = new int[mid];
        int right[] = new int[n - mid];
        System.arraycopy(arr, 0, left, 0, mid);
        System.arraycopy(arr, mid, right, 0, n - mid);
        mergeSort(left);
        mergeSort(right);
        merge(arr, left, right);
    }

    public static void merge(int[] arr, int[] right, int[] left) {
        int i = 0; //left
        int j = 0; //right
        int k = 0; //arr
        while (i < left.length && j < right.length) {
            if (left[i] <= right[j]) {
                arr[k] = left[i];
                k++;
                i++;
            } else {
                arr[k] = right[j];
                k++;
                j++;
            }
        }
        while (i < left.length) {
            arr[k] = left[i];
            k++;
            i++;
        }
        while (j < right.length) {
            arr[k] = right[j];
            k++;
            j++;
        }
    }
}
```

2. Binary Search (البحث الثنائي)/

أ. فكرة الخوارزمية:

تعمل فقط على مصفوفة مرتبة

تقارن القيمة المطلوبة مع العنصر الأوسط في المصفوفة

إذا كانت القيمة أكبر، تبحث في النصف الأيمن، وإذا كانت أصغر، تبحث في النصف الأيسر

تتكرر العملية حتى يتم العثور على العنصر أو استنفاد جميع الاحتمالات

ب. تطبيق الخوارزمية يدويًا للبحث عن الرقم ٧ في المصفوفة المرتبة

[1, 3, 4, 6, 7, 8, 10, 13, 14]:

الخطوات:

1. المصفوفة مرتبة بالفعل.

2. البحث:

- [النطاق: 1, 3, 4, 6, 7, 8, 10, 13, 14]

- (العنصر الأوسط = 7) (المؤشر 4)

- تم العثور على العنصر المطلوب $7 == 7 \rightarrow$

ج. تحليل التعقيد الزمني:

- لأن المصفوفة تنقسم إلى نصفين في كل خطوة ($O(\log n)$): تعقيد البحث -

- سبب التعقيد: عدد الخطوات يتناقص بشكل لوغاريتمي مع حجم المصفوفة -

تطبيق الخوارزمية عمليا بكود بلغة الجافا /

```
// Deema Mohammed AL-Maqadma
public class BinarySearch {
    public static void binarySearch(int[] arr, int target) {
        int left = 0; // ترتيب العنصر الأول
        int right = arr.length - 1; // ترتيب العنصر الأخير
        boolean isFaound = false;

        while (left <= right) {
            int mid = left + (right - left) / 2; // ترتيب العنصر الاوسط

            if (arr[mid] == target) {
                isFaound = true;
                break;
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        if (isFaound) {
            System.out.println("Target : " + target + "is Faound");
        } else {
            System.out.println("Target : " + target + "is NOT Faound");
        }
    }
}
```

السؤال الثاني:

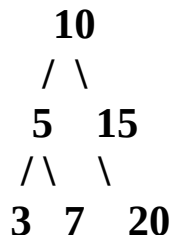
(Tree Traversal) خوارزمية اجتياز الشجرة

1. Post-order Traversal (الاجتياز البعدي)

أ. فكرة الخوارزمية:

- 1. (تزرع العقدة اليسرى أولاً، ثم العقدة اليمنى، ثم العقدة الحالية (الجذر).
- 2. تُستخدم لحذف الشجرة أو لحساب التعبيرات الرياضية في أشجار التعبير.

ب. تطبيق الخوارزمية يدوياً على الشجرة المعطاة:



الخطوات:

1. (ابدأ من الجذر (10)، انتقل إلى اليسار (5).
2. من (5)، انتقل إلى اليسار (3) ← تطبع 3.
3. ارجع إلى (5)، انتقل إلى اليمين (7) ← تطبع 7.
4. ارجع إلى (5) ← تطبع 5.
5. انتقل إلى اليمين من الجذر (10)، انتقل إلى اليمين (15) ← تطبع 15.
6. ارجع إلى (10) ← تطبع 10.
7. أخيراً، الجذر (10) ← تطبع 10.

[النتيجة: 3، 7، 5، 15، 10]

ج. تحليل التعقيد الزمني:

- 1. لأن كل عقدة تُزار مرة واحدة فقط ($O(n)$): التعقيد -

"واخر دعواهم ان الحمد لله رب العالمين"